Week 2 - Introduction to QTA

Dino Wildi

22 October 2026

What is GitHub?

Online tool to share, publish, and control code
Main functions we use in computational social science:

- Save and version control code online
- Share with collaborators (or between machines)
- Publish code and data to the world

GitHub Lingo

- Repository (Repo): a collection of files
- Commit: A snapshot of a repo at a point in time
- Clone: Downloading a repo to your computer
- Push/Pull: Uploading/downloading commits
- Branch: A parallel copy of a repo
- Merge: Combining two branches into one version

How To Use GitHub

- Use GitHub Desktop (alternative: command line)
- Clone a repo to your machine and work locally
  - To work with course data: do NOT push your changes
  - To work with your own data: regularly push changes
- Keep clone and original repo as synced as possible
- WARNING: Pulling new commits will overwrite local changes

Why Study Texts?

- Large body of political text produced continuously
- Often produced by actors of interest themselves
- Direct interaction with other actors (e.g. voters)
- Much richer information than e.g. survey data

How to Study Texts?

Suppose you want to study patriotism in US Presidential speeches.
What could you do?

How to Study Texts?

Suppose you want to study patriotism in US Presidential speeches.
What could you do?

- Close reading of texts (similar to historic source analysis)
- Comparative reading of specific texts
- Discourse/framing analysis: How is patriotism mentioned?
- Hand-coding texts (e.g. defining some "patriotism index")

How to Study Texts?

Suppose you want to study patriotism in US Presidential speeches.
What could you do?

- Close reading of texts (similar to historic source analysis)
- Comparative reading of specific texts
- Discourse/framing analysis: How is patriotism mentioned?
- Hand-coding texts (e.g. defining some "patriotism index")

These methods are **detailed**, but **scale rather poorly**

The Quantitative Promise

- Find a way to transform text to numbers automatically
- Analyze the resulting numbers rather than the texts
- Can be scaled significantly further (possibly near infinite)
- Allows measures of certainty (standard errors, significance etc)
- *Potentially* allows removal/reduction of language barriers

The cost for this: we sacrifice detail and contextual understanding for scale and more specific measures

Ways of Transformation

We can transform text in multiple ways:

- Count words per document and use those numbers
- Measure properties of the words themselves
- Classify texts into some kind of category
- Use some meaning representation (e.g. embeddings, LSS)

Depending on what you measure, you drop different information!

Types of Jobs

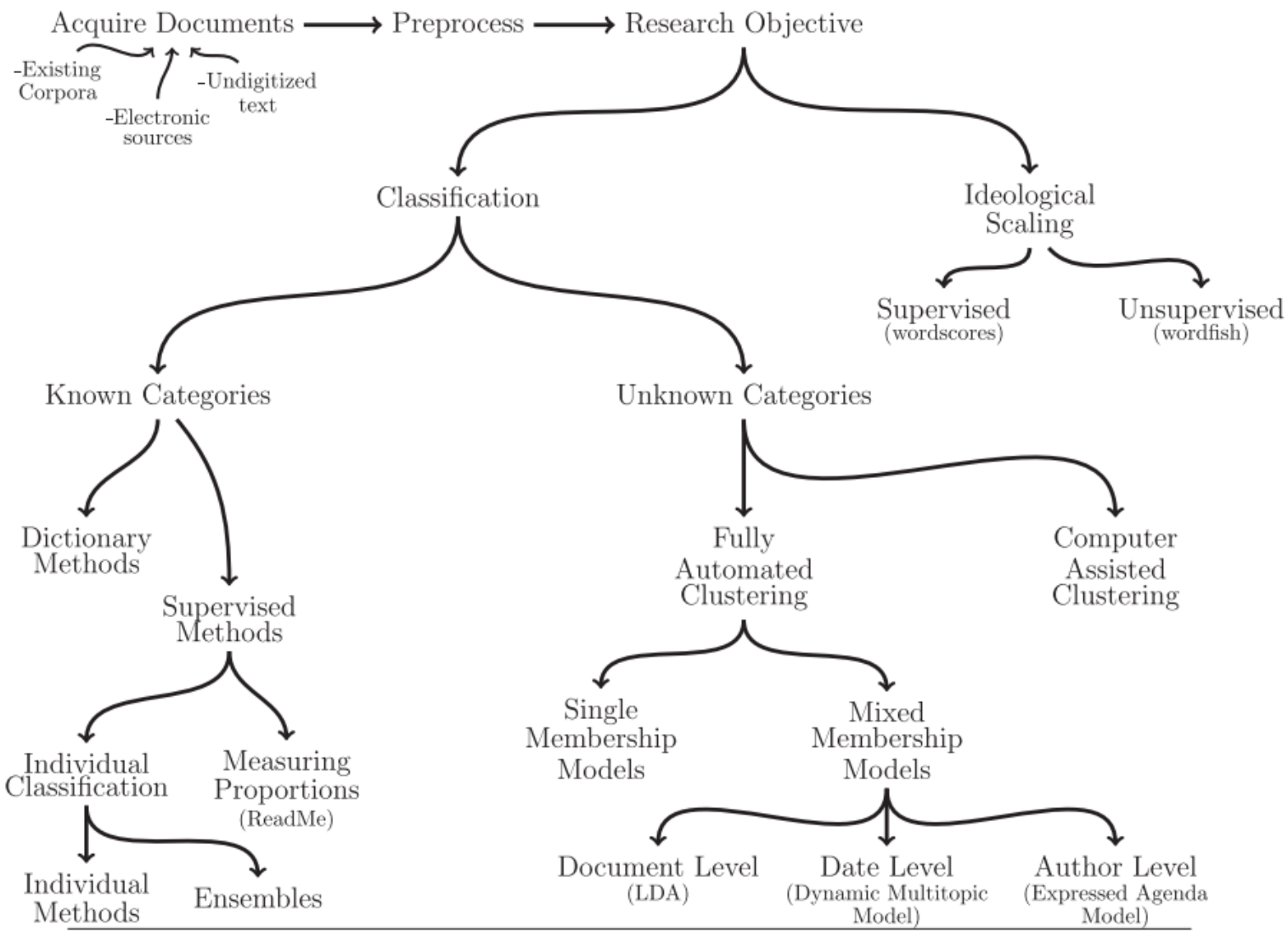


Figure 1: Figure from Grimmer & Stewart (2013)

Zollinger 2024: Cleavage Identities

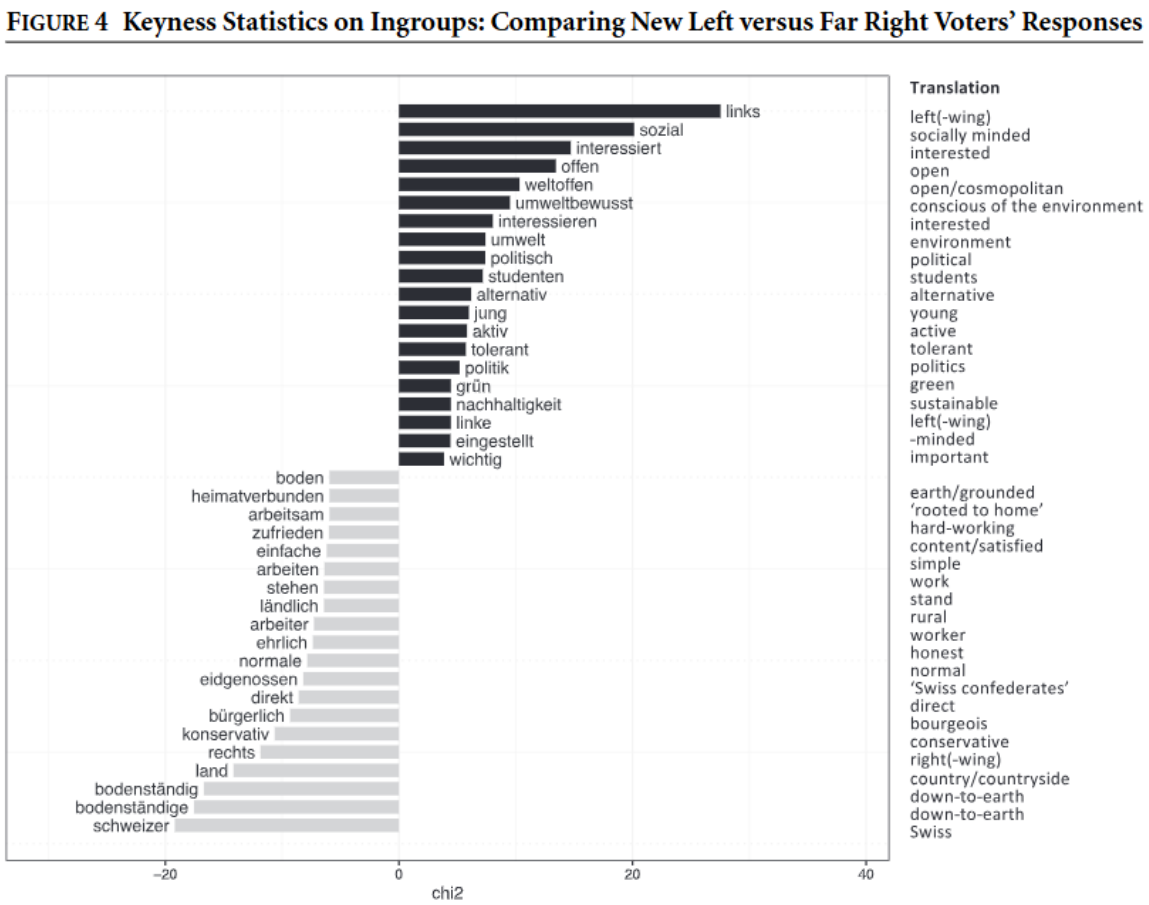


Figure 2: Figure from Zollinger (2024)

Müller 2021: Temporal Focus of Campaign Communication

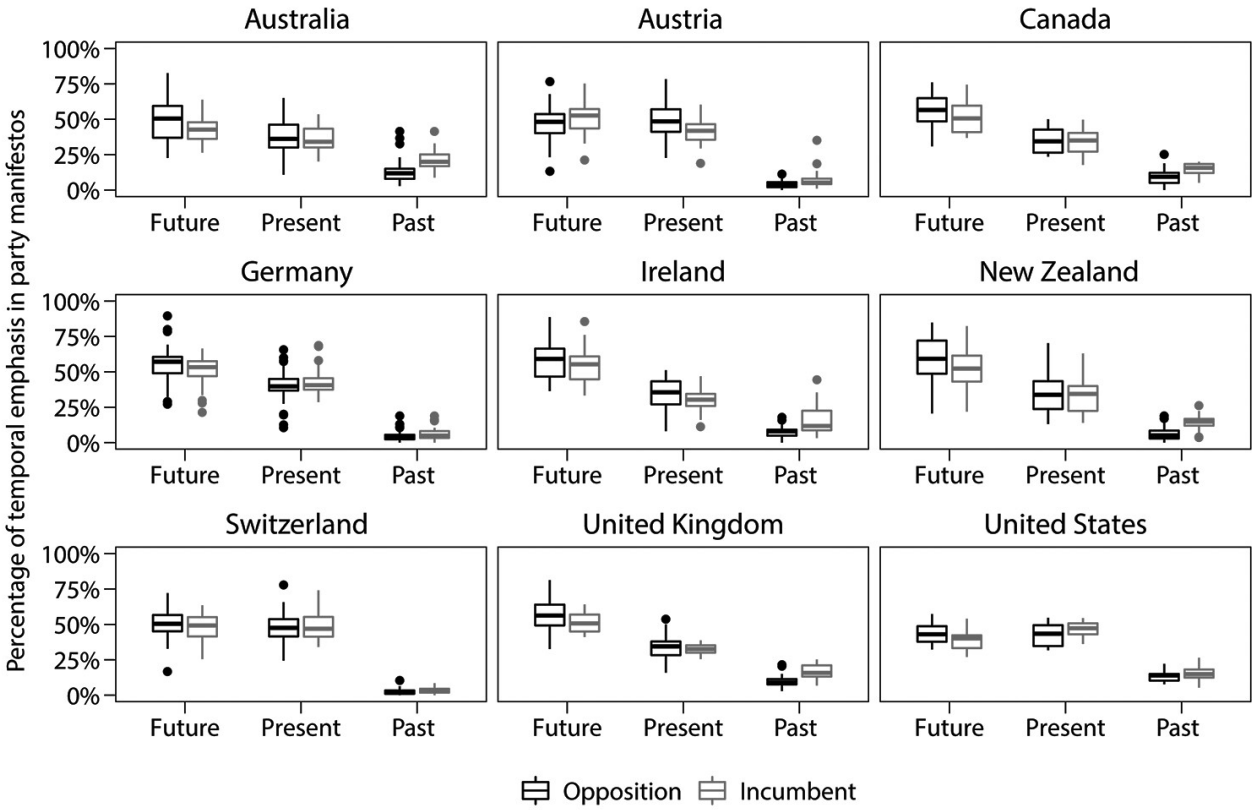


Figure 3: Figure from Müller (2021)

Wildi (forthcoming): RRP strategy under threat

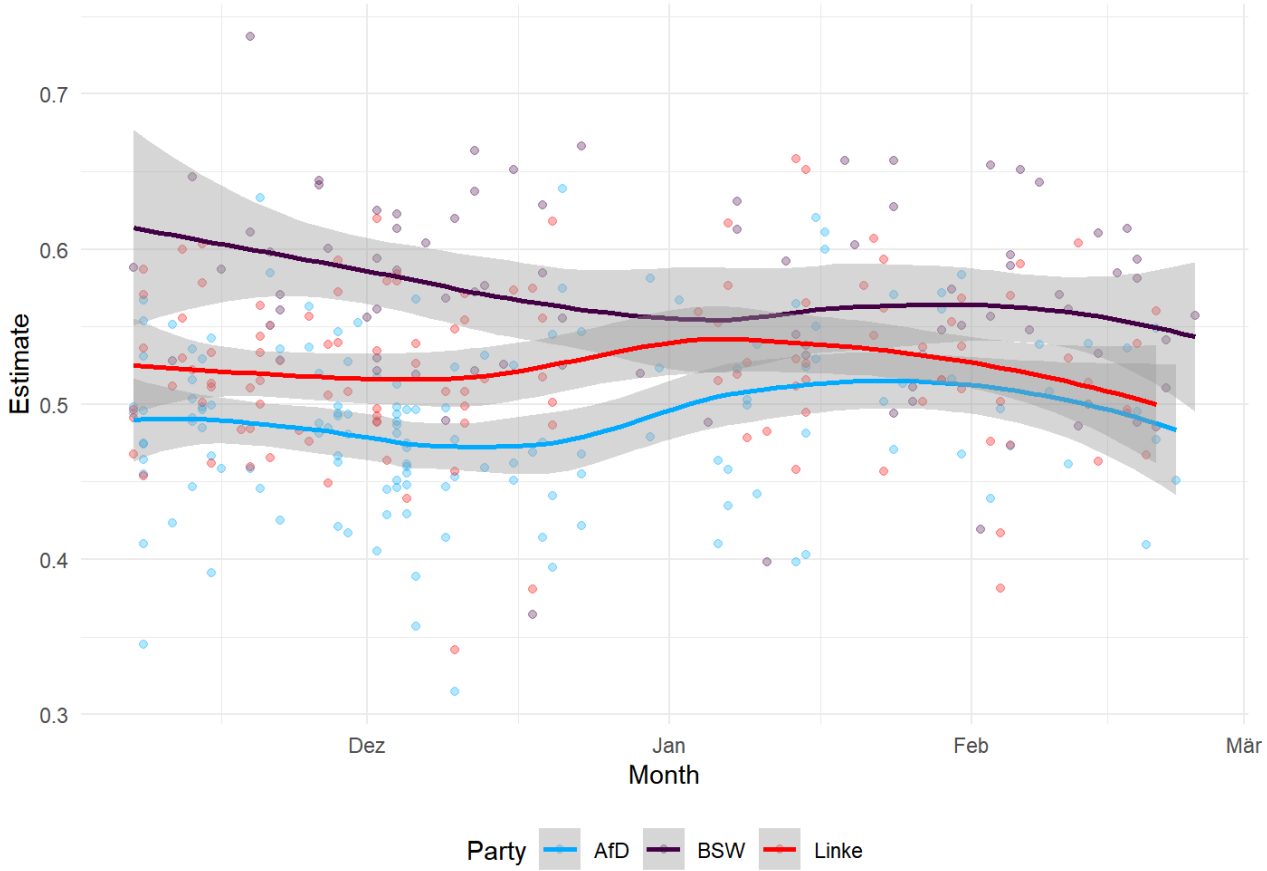


Figure 4: Figure from Wildi (forthcoming)

Models don't represent text completely

- We do not know what actually goes into a text
- QTA represents the text **as it is:**
  - Good for descriptions of how words are used
  - Bad for causal inferences or statements about data generation
- We often need some further analysis for causal hypotheses
- Think critically about what your model actually represents!

Know what you want

- Define your quantity of interest
- Define your task: describe, classify, extract, scale...
- Know your texts reasonably well
- Have an expectation of what you want to see

Good rule of thumb: If you can't perform the task yourself, you can't assess a model's performance on it!

Methods are task-specific

Every method has benefits and drawbacks:

- Computational power
- Human workload for training and validation
- Control and interpretability of the output
- Reproducibility (this is related to control)

Always be able to defend your choices!

Validation is everything

Traditional quants (e.g. regression): You know what you get
QTA: You don't get a standard interpretation of measures
How do you test what you measured?

Validation is everything

Traditional quants (e.g. regression): You know what you get
QTA: You don't get a standard interpretation of measures
How do you test what you measured?

- Face validity: Does it make sense?
- Check key words: Are they representative?
- Comparing to expectations: Are there red flags?
- Gold standard data: Does the model agree with you?

Text choice matters

Texts come with a context: you write differently in an email to me than in a text to your friend

- Cross-domain transfer is difficult
- Training data and live data needs to match
- Pre-processing can make a big difference (see Session 12)
- Pay particular attention to reference texts for scaling

Starting a Markdown project

- Ensure you have installed "rmarkdown" and "knitr"
- A correction to last week: we will use **Quarto**
- Click File > New File > Quarto Document
- Fill in title + Author; leave rest the same
  - Untick visual editor if you prefer; matter of taste
- Recommended: Settings to "Preview in Viewer Pane"

The Markdown Language

- Use hashtags to structure text with titles
- Use Asterisks for **italics** and ****bold text****
- Insert code chunks using the command ```` ```{r} ````
- End code chunks with ```` ``` ````
- Code options get inserted with `#|`:
  - `#| echo: false` = only print output
- Full list of code options here

Rendering Quarto

- Click on "Render" in the bar over the script window
- Shows rendered document in the Viewer pane
- Creates rendered file in the same folder
- You can submit the rendered HTML file as your lab assignment

What is Tidyverse?

Tidyverse is a suite of 9 packages loaded at once:

- Data loading: readr
- Data wrangling: dplyr
- Reshaping data: tidyr
- Tibbles (see last week's code): tibble
- Data visualisation: ggplot2
- Handling strings (i.e. characters): stringr
- Handling dates: lubridate
- Don't worry about it now: purrr, forcats

Tidy Data

How would you organise "clean" data?

Tidy Data

How would you organise "clean" data?
"Tidy" data is a framework to organise data:

- Each variable is its own column
- Each observation is its own row
- Each cell is one specific value (and nothing else)

Keeping your data tidy makes it easier to use functions

Reshaping data

Most data frames are larger than needed – use dplyr to solve:

- `select(ds, name)` to select columns by name
- `filter(ds, col == value)` to filter for row values
- `rename(ds, newname = oldname)` to rename columns
- `arrange(ds, column)` to sort by a column
- `distinct(ds)` to drop duplicated values

Pivoting data

Data can be **long** or **wide** depending on how multiple instances of the same observation are treated.

Imagine you have a dataset of countries and years:

- **Long** data has every year-country-combo be its own row
- **Wide** data: Every country in a row, every year in a column
- Most of the time we want long data
- Wide data is often easier to collect and look at
- Use `pivot_longer()` and `pivot_wider()` to pivot

Creating new variables

We often want to add new variables (e.g. a mean or a tag):

- `mutate(ds, newvar = )` creates a new variable
- Entering a value just inserts that value (e.g. for tags)
- Entering a function inserts the result of that function
- Using `if_else()` or `case_when()` allows conditional values
- Use `mutate(ds, across())` to change multiple variables

Summarising data

- `summarise(ds, var = fn())` summarises a variable
- Example: `summarise(mtcars, mpg = mean(mpg))`

Summarising data

- `summarise(ds, var = fn())` summarises a variable
- Example: `summarise(mtcars, mpg = mean(mpg))`
- `group_by(ds, column)` groups data by a column
- Calling `summarise()` on grouped data applies the function per group
- Example:
  `group\_by(mtcars, cyl) |> summarise(mpg = mean(mpg))`